8.12 DMA 控制方式

考纲定位：计算机组成原理 · 第 8 章 I/O 系统（8.12）｜重要度 ★★★★｜常见考法：选择题（DMA 流程、DREQ/DACK、HOLD/HLDA、三种争用方式、DMA vs 中断），高频考点

一句话核心：DMA 用 DMAC 接管总线，在设备与主存之间成块搬运数据；CPU 只做初始化和善后

一、定义与控制步骤

直接存储器存取（DMA）方式

■ 在DMA方式中，引入了一个**独立的硬件模块**，即**DMA控制器DMAC**。

□ 在进行大数据块传输时，DMAC会临时向CPU“借用”总线的控制权，之后由DMAC直接控制数据在I/O设备与主存（内存）之间流动，不再经过CPU的寄存器。

■ DMA方式的传输过程：

① CPU访问DMAC接口以设置DMA传输参数（如主存地址、数据块长度、传输方向等）。

② CPU向I/O设备的I/O接口发送命令和参数来启动I/O设备。

③ 传输过程中CPU可以继续执行其他程序。

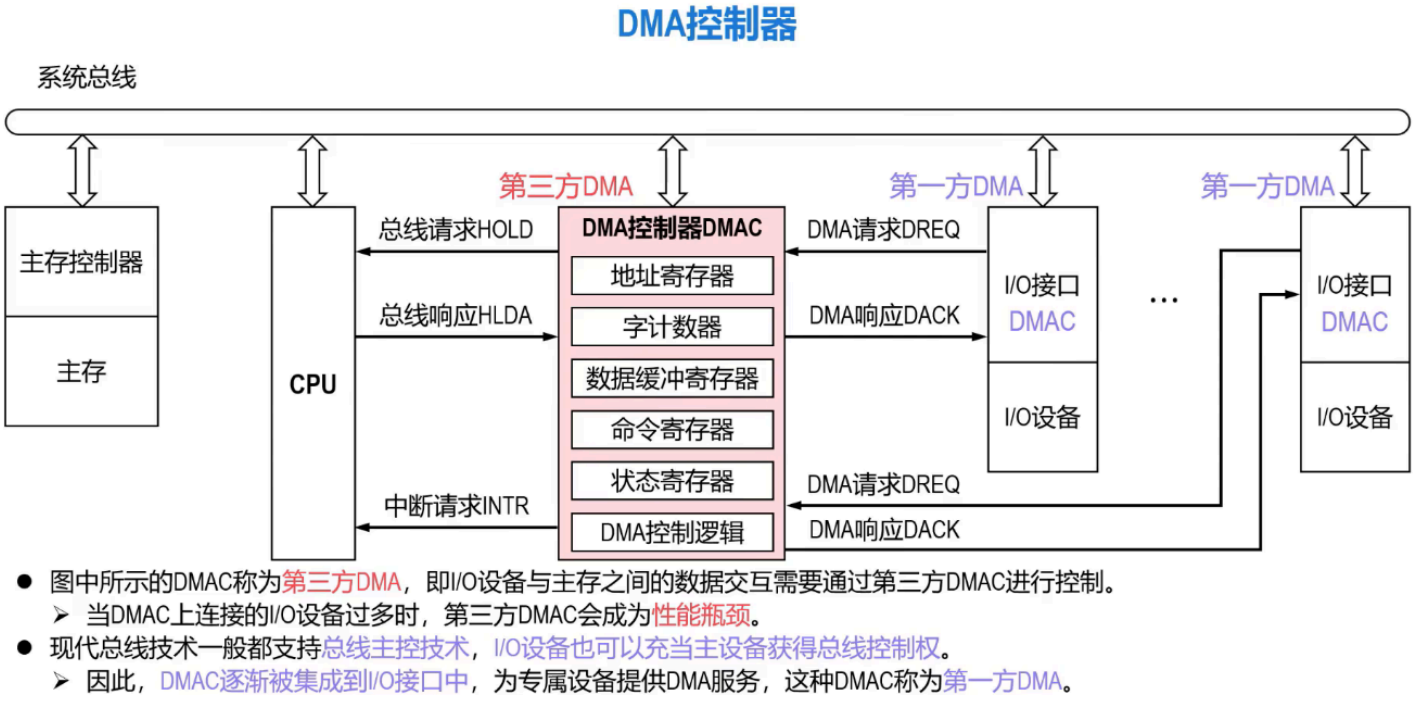
④ I/O设备需要进行DMA操作时，由DMAC向CPU申请总线控制权用于数据传输。

⑤ 一个数据块传输结束后，DMAC会通过中断方式请求CPU对数据缓冲区和DMAC进行后续处理。

DMA是计算机系统中一种重要的性能优化技术，它通过硬件接管数据传输任务，解决了I/O操作中CPU开销过大的瓶颈问题。它使得CPU从繁重的数据搬运工作中解脱出来，更加专注于计算任务，是现代计算机能够高效处理大量I/O操作（如视频播放、文件读写、网络通信）的关键基础之一。

- DMA 方式引入一个独立的硬件模块，即 DMA 控制器（DMAC）
- 在进行大数据块传输时，DMAC 会临时向 CPU“借用”总线控制权，由 DMAC 直接控制数据在 I/O 设备与主存之间流动，不再经过 CPU 的寄存器
- DMA 传送过程
  1. CPU 访问 DMAC 接口，设置 DMA 传送参数（如主存起始地址、数据块长度、传送方向等）
  2. CPU 向 I/O 设备的 I/O 接口发送命令和参数，启动 I/O 设备
  3. 传送过程中 CPU 可以继续执行其他程序
  4. I/O 设备需要进行 DMA 操作时，向 DMAC 申请总线控制权，用于数据传输
  5. 一个数据块传输结束后，DMAC 通过中断方式请求 CPU 对数据缓冲区和其他 DMAC 进行后续处理
- DMA 是计算机系统中重要的性能优化技术：它通过硬件接管数据传输任务，解决 I/O 操作中 CPU 开销过大的问题，使 CPU 从繁重的数据搬运中解脱出来，更专注于计算任务

二、DMA 控制器的结构



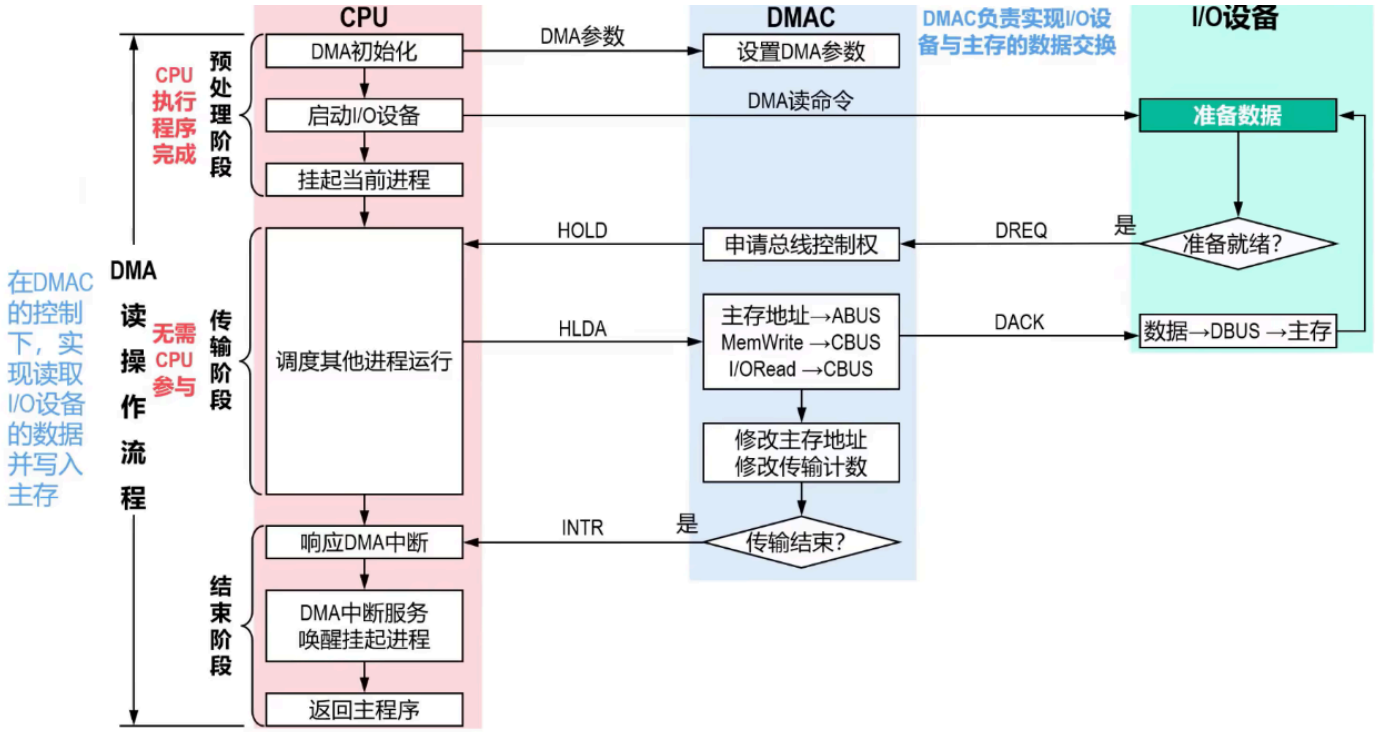
- 图中所示的DMAC称为第三方DMA，即I/O设备与主存之间的数据交互需要通过第三方DMAC进行控制。
  - 当DMAC上连接的I/O设备过多时，第三方DMAC会成为性能瓶颈。
- 现代总线技术一般都支持总线主控技术，I/O设备也可以充当主设备获得总线控制权。
  - 因此，DMAC逐渐被集成到I/O接口中，为专属设备提供DMA服务，这种DMAC称为第一方DMA。

DMAC 内部的核心部件：

部件	作用
主存地址计数器	存放当前要访问的主存地址；每传送一个字自动加 1
传送长度计数器	存放剩余传送字数；每传送一个字自动减 1，减到 0 表示块结束
数据缓冲寄存器	暂存传送的数据，协调设备与主存的速度差异
DMA 请求触发器	记录设备发来的 DMA 请求
控制/状态逻辑	产生控制信号，管理总线请求/应答、传送方向、结束判断
中断机构	块传送结束后向 CPU 发中断请求

- 第三方 DMA：图中的 DMAC 位于 I/O 设备与主存之间，I/O 设备的数据交互需要通过第三方 DMAC 控制；当 DMAC 上连接的 I/O 设备过多时，第三方 DMAC 会成为性能瓶颈
- 第一方 DMA：现代总线技术一般支持总线主控，I/O 设备也可以作为主设备获取总线控制权；DMAC 逐渐被集成到 I/O 接口中，为专属设备提供 DMA 服务，这种 DMAC 称为第一方 DMA
- 408 选择题常问“DMAC 包含哪些部件”：PC、PSW、通用寄存器属于 CPU，不属于 DMAC

三、DMA 控制的流程图



三列（CPU / DMAC / I/O 设备）代表三方参与者，左侧竖排三个阶段（预处理、DMA 传输、结束）代表时间线。一句话立意：CPU 只负责“头”（初始化）和“尾”（善后），中间的数据搬运完全由 DMAC 硬件接管——这也是 DMA（直接存储器存取）名字的由来

3.1 预处理阶段（CPU 执行，软件完成）

CPU 干三件事：

1. DMA 初始化：向 DMAC 写入三个参数——主存起始地址、传送字数（块长）、传送方向。这些参数落进 DMAC 内部的主存地址计数器、传送长度计数器、控制寄存器
2. 启动 I/O 设备：向设备发启动命令（图中“DMA 读命令”，即通知设备开始准备数据）
3. 挂起当前进程：从操作系统视角，发起 I/O 的进程阻塞等待，CPU 去调度其他进程运行

3.2 DMA 传输阶段（无需 CPU 参与，硬件完成）

这是全图的核心，DMAC 接管一切：

1. 设备“准备就绪”→ 向 DMAC 发 DREQ（设备请求 DMA 传送）
2. DMAC 向 CPU 发 HOLD，申请总线控制权
3. CPU 回 HLDA，让出总线（注意：此时 CPU 只是不访存了，仍可调度其他进程执行不需要访存的部分）
4. DMAC 成为总线主人，行使类似 CPU 的职能：主存地址→ABUS，MemWrite→CBUS，I/ORead→CBUS——同时给主存“写命令”、给设备“读命令”，并回 DACK 通知设备
5. 设备把数据放上 DBUS，直达主存——数据全程不经过 CPU
6. DMAC 硬件自动修改主存地址（+1）、修改传输计数（-1）
7. “传输结束？”判断：否 → 回到第 4 步传下一个字；是 → 进入结束阶段

3.3 结束阶段（CPU 执行，软件完成）

1. DMAC 发 INTR（中断请求）——DMA 结束时也要走一遍完整的中断流程
2. CPU 响应 DMA 中断，执行 DMA 中断服务程序：校验数据是否正确、判断是否需要继续传送下一块、唤醒挂起的进程
3. 返回主程序

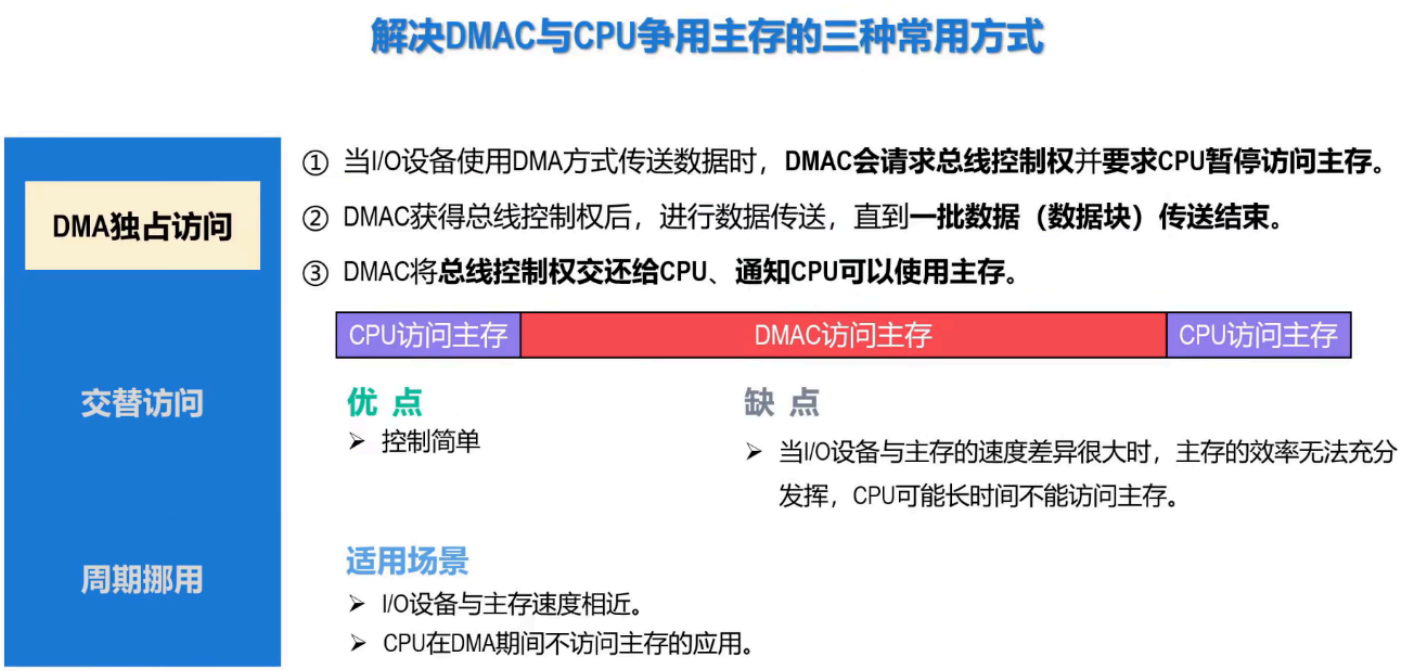
3.4 信号线配对辨析

信号	谁发给谁	含义
DREQ / DACK	设备 ↔ DMAC	设备申请 DMA 传送，DMAC 应答
HOLD / HLDA	DMAC ↔ CPU	DMAC 申请总线使用权，CPU 让出总线
INTR	DMAC → CPU	整块传完，请 CPU 来善后

记忆口诀：“设备找 DMAC（DREQ/DACK），DMAC 找 CPU（HOLD/HLDA），传完叫 CPU（INTR）”——三级接力，别把 HOLD 和 DREQ 搞混

四、解决 DMA 与 CPU 争用主存的三种方式

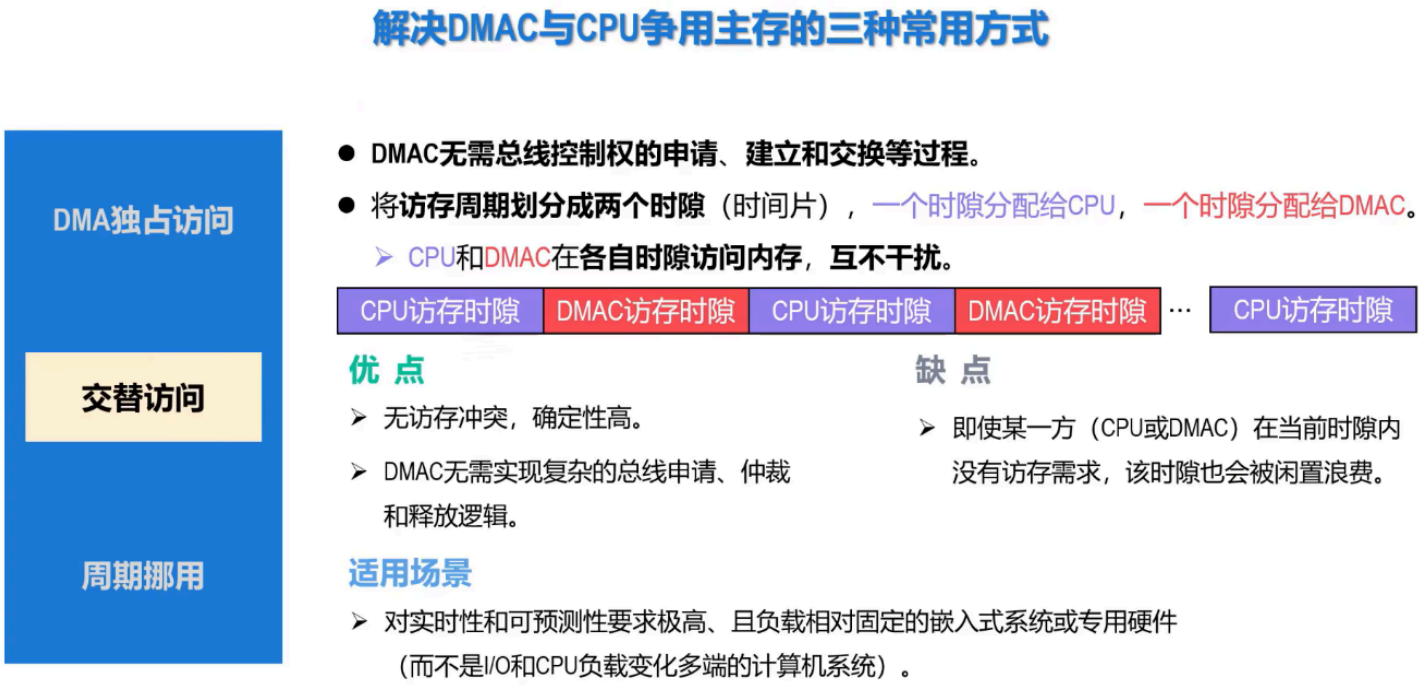
4.1 DMA 独占访问



- 当 I/O 设备使用 DMA 方式传送数据时，DMAC 会请求总线控制权并要求 CPU 暂停访问主存
- DMAC 获得总线控制权后，进行数据传送，直到一批数据（数据块）传送结束
- DMAC 将总线控制权交还给 CPU，通知 CPU 可以使用主存
- 优点：控制简单
- 缺点：当 I/O 设备与主存的速度差异很大时，主存的效率无法充分发挥，CPU 可能长时间不能访问主存

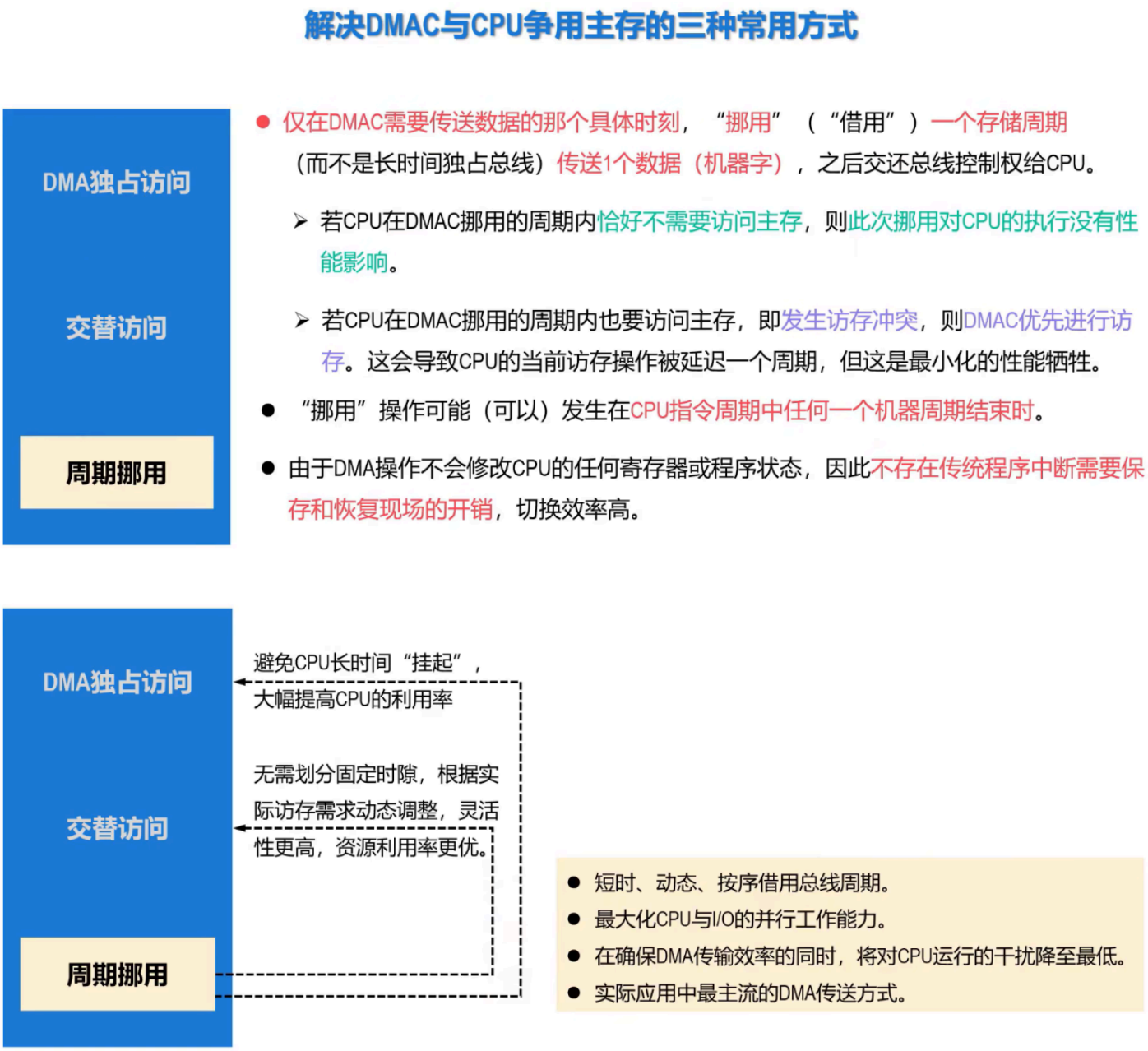
- 适用：I/O 设备与主存速度相近；CPU 在 DMA 期间不访问主存的应用

4.2 交替访问



- DMAC 无需总线控制权的申请、建立和交换等过程
- 将访存周期划分成两个时间片，一个时间片分配给 CPU，一个时间片分配给 DMAC，CPU 和 DMAC 在各自时隙访问内存，互不干扰
- 优点：无访存冲突，确定性高；DMAC 无需实现复杂的总线申请、仲裁和释放逻辑
- 缺点：即使某一方（CPU 或 DMAC）在当前时隙内没有访存需求，该时隙也会被闲置浪费
- 适用：对实时性和可预测性要求高、负载相对固定的嵌入式系统或专用硬件，而不是 I/O/CPU 负载变化多端的通用计算机系统

4.3 周期挪用（周期窃取）



- 仅在 DMAC 需要传送数据的那个具体时刻，“挪用”（“借用”）一个存储周期（而不是长时间独占总线）传送 1 个数据（机器字），之后交还总线控制权给 CPU
- 若 CPU 在 DMAC 挪用的周期内恰好不需要访问主存，则此次挪用对 CPU 的执行没有影响
- 若 CPU 在 DMAC 挪用的周期内也需要访问主存，即发生访存冲突，则 DMAC 优先进行访存，这会导致 CPU 的当前访存操作被延迟一个周期，但这是最小化的性能牺牲
- “挪用”操作可能发生在 CPU 指令周期中任何一个机器周期结束时
- 由于 DMA 操作不会修改 CPU 的任何寄存器或程序状态，因此不存在传统程序中断中需要保存和恢复现场的开销，切换效率高
- 优点：避免 CPU 长时间“挂起”，大幅提高 CPU 利用率；无需划分固定时隙，根据实际访存需求动态调整，灵活性更高，资源利用率更优；短时、动态、按需借用总线周期，最大化 CPU 与 I/O 的并行工作能力；在确保 DMA 传输效率的同时，将对 CPU 运行的干扰降至最低
- 实际应用中最主流的 DMA 传送方式

4.4 三种方式对比

方式	总线控制	对 CPU 的影响	适用
DMA 独占访问	DMAC 整块独占总线	CPU 长时间不能访存	设备与主存速度相近、CPU 期间不访存
交替访问	固定时隙交替	无冲突，但时隙可能闲置	实时性/确定性要求高的嵌入式系统
周期挪用	按需借一个存储周期	CPU 访存最多延迟一个周期	通用计算机，最主流

五、DMA 方式与中断驱动方式的对比

DMA方式与中断驱动方式的对比

对比维度	中断驱动方式	DMA方式
请求-应答机制	请求的是CPU时间。 响应时机在指令周期结束时刻。	请求的是总线控制权。 响应时间在一个总线事务完成后。
数据传送实现方式	通过CPU执行程序实现数据传送。 需要保护和恢复程序执行现场，影响系统性能。	通过额外的硬件（DMAC）实现数据传送。 不改变CPU现场，不影响系统性能。
主要应用	处理各种随机事件。	大量、高速的数据传输。

- DMA方式是中断驱动方式的进一步发展：
  - 在数据传输的核心阶段（数据块传送），通过DMAC接管总线控制权来“解放”CPU的参与，从而实现了更高的效率。
  - 但在传输的开始阶段，仍需要CPU参与设置DMA参数；在结束阶段，DMAC会通过DMA中断请求通知CPU。

对比维度	中断驱动方式	DMA 方式
请求/应答机制	请求的是 CPU 时间	请求的是总线控制权
响应时机	指令执行完毕时	一个总线事务完成后/存取周期结束
数据传送实现	CPU 执行程序完成数据传送	通过专门的硬件（DMAC）实现数据传送
对 CPU 现场	需要保护和恢复现场，影响系统性能	不改变 CPU 现场，不影响系统性能
主要应用	处理随机事件	大量、高速的数据传输

- DMA 是中断驱动方式的进一步发展：在数据传输的核心阶段（数据块传送），通过 DMAC 接管总线控制权来“解放”CPU 的参与，从而实现更高的效率
- 但在传输的开始阶段，仍需要 CPU 参与设置 DMA 参数；在结束阶段，DMAC 会通过 DMA 中断请求通知 CPU

六、易错点清单

- **易错 1：“DMA 方式不需要 CPU 参与”。** 错。只有数据传送阶段不需要 CPU，预处理（初始化）和结束（中断善后）都需要 CPU 执行程序。准确说法是“数据传送不经过 CPU”，而非“全程无 CPU”
- **易错 2：混淆 DMA 与中断的响应时机。** 程序中断必须等一条指令执行完；DMA 请求在一个存取周期（机器周期）结束后即可响应——它可以插进指令周期的中间。这是“DMA 优先级更高”在时序上的体现
- **易错 3：认为 DMA 方式没有中断。** 错。图中的 INTR 就是 DMA 中断——每个数据块传完后 DMAC 主动申请一次中断，让 CPU 做善后。区别在于中断频率：程序中断逐字中断，DMA 整块只中断一次
- **易错 4：以为数据要经过 DMAC 的寄存器中转给 CPU。** 数据通路是设备 → DBUS → 主存，不经过 CPU 的任何寄存器。DMAC 只提供地址（ABUS）和控制信号（CBUS），数据自己走数据总线
- **易错 5：认为修改地址和计数是 CPU/软件做的。** 是 DMAC 硬件自动完成的——这正是 DMA“硬件搬运”的本质，软件从头到尾没碰过数据
- **易错 6：认为 DMA 期间 CPU 完全停机。** 只在“停止 CPU 访问”方式下成立。周期挪用时 CPU 只是让出个别存取周期，其余时间照常执行程序；即使不访存的部分（如缓存命中的指令）也可继续运行
- **易错 7：DMA 读/写的命名陷阱。** 唐朔飞教材与 408 按主存角度命名：DMA 读 = 主存→设备，DMA 写 = 设备→主存。而本图标注的“DMA 读命令”是从设备角度（读设备入主存，对应主存角度的“DMA 写”——注意图里发给主存的是 MemWrite）。记数据流向比背名词可靠，读题时先确认主存是收还是发
- **易错 8：DMAC 组成部件的选择题。** DMAC 内部包含：主存地址计数器、传送长度计数器、数据缓冲寄存器、DMA 请求触发器、控制/状态逻辑、中断机构。PC、PSW、通用寄存器属于 CPU，不属于 DMAC——选择题常混入干扰
- **易错 9：三种争用方式的适用场景记反。** 独占访问适合“设备与主存速度相近、CPU 期间不访存”；交替访问适合“实时性/确定性要求高的嵌入式系统”；周期挪用是通用计算机的主流

一句话收束：预处理 CPU 交钥匙（地址/长度/方向），传输 DMAC 当司机（申请总线、直达主存、自动改指针），结束 CPU 来收尾（一次中断、校验唤醒）——DMA 的全部价值就是把“逐字打断 CPU”变成“整块不打扰 CPU”，而它自己本质上就是一个会抢总线、会计数的简化版 CPU

七、考法透视

常见题型

- 给 DMA 流程图，补全 DREQ/DACK、HOLD/HLDA、INTR 的方向与作用
- 判断某个阶段是否需要 CPU 参与
- 比较 DMA 与中断的响应时机、数据通路、现场保护
- 判断三种争用方式的优缺点与适用场景
- 给 DMAC 组成部件，选出不属于 DMAC 的部件

常见设错点

1. 认为 DMA 全程不需要 CPU
2. 认为 DMA 不会产生中断
3. 认为数据经过 DMAC 寄存器中转
4. 认为地址/计数由 CPU 修改
5. 认为 DMA 响应在指令执行完毕时

6. 认为 DMA 期间 CPU 一定完全停机
7. 把 DMA 读/写的方向记反
8. 把 PC、PSW 算进 DMAC

八、知识关联

- 前置：8.4 概述（控制方式全景）、8.6 中断驱动方式概述（中断方式的局限）、8.9 中断请求（DMA 结束用中断善后）
- 后续：通道方式、I/O 处理机（DMA 的进一步发展）
- OS 视角：进程阻塞/唤醒、DMA 与中断在驱动中的配合
- 全章导航：8.0 导航与术语

九、费曼自测

1. DMA 的三个阶段分别由谁主导 → 预处理 CPU 软件；DMA 传输 DMAC 硬件；结束 CPU 软件（中断善后）
2. DREQ/DACK 与 HOLD/HLDA 分别是谁和谁握手 → DREQ/DACK 是设备与 DMAC；HOLD/HLDA 是 DMAC 与 CPU
3. DMA 与中断的响应时机有什么不同 → 中断在指令执行完毕的边界响应；DMA 在一个存取周期/总线事务结束后即可响应，可以插进指令周期中间
4. 三种解决 DMA 与 CPU 争用主存的方式是什么 → DMA 独占访问、交替访问、周期挪用（周期窃取）
5. 为什么说 DMA 并没有“全程不需要 CPU” → 数据传送阶段不需要 CPU，但预处理初始化参数、结束阶段中断善后都需要 CPU